



13. Convolutional Neural Networks

"Looking beyond the pixels to find the picture."

Previous Content:

[🔗 12. Introduction to TensorFlow & Keras](#)

Contents:

- [1. Introduction to Neural Network for Computer Vision](#)
 - [1.1. Before Deep Learning: The Era of Rules](#)
 - [1.2. The Shift: Learning from Data](#)
- [2. The Basics of Convolutional Neural Networks \(CNNs\)](#)
 - [2.1. The Convolutional Part: Extracting Features](#)
 - [2.2. Understanding the Convolution Operation](#)
 - [2.3. The Neural Network Part](#)
- [3. Pooling Layers in Convolutional Neural Networks \(CNNs\)](#)
 - [3.1. Max Pooling](#)
 - [3.2. Average Pooling](#)
- [3. The Workflow of a Convolutional Neural Network](#)
 - [3.1. The Convolution Operation by Hand](#)
 - [3.2. Feature Extraction on MNIST](#)
 - [3.3. Filtering, Response Maps, and Feature Maps](#)
 - [3.4. Training the Convolutional Neural Network](#)
- [4. Image Classification with Convolutional Neural Networks](#)
 - [4.1. Cat vs Dog Dataset \(CIFAR-10 subset\)](#)
 - [4.2. Baseline Convolutional Model](#)
 - [4.3. Improving the Model: Augmentation + Dropout](#)

[Summary](#)

[References](#)

Throughout this course, we have explored Neural Networks from their structure to their applications, and we have learned how to improve, validate, and adapt them for different problems. But early on, specifically right at the introduction to the entire course, I mentioned something important: **deep learning is powerful because it can automatically extract features from raw data**. Until now, that idea may have felt abstract.

In this section, we finally see that idea come alive through Computer Vision, using Convolutional Neural Networks (CNNs). This is where neural networks begin to “see” patterns inside images rather than relying on manually engineered features.

Before we go further, we need to remember a fundamental rule: *Neural networks always take tensors as input*. That means every image must be transformed into a tensor with a specific structure before the model can process it.

So the real question becomes: *how do we convert an image into a format a neural network can understand?* That is what we will uncover next.

1. Introduction to Neural Network for Computer Vision

Computer Vision is the study of enabling machines to understand and interpret images. While this field is large and complex (and will be explored in more detail in its own course), for now I want us to focus on how it evolved—and why that evolution naturally leads us to Convolutional Neural Networks.

1.1. Before Deep Learning: The Era of Rules

Before deep learning became popular, people tried to build image recognition systems using hand-crafted rules.

The idea was simple:

- Collect a dataset of images
- Manually design rules that describe what a “cat” or a “dog” looks like
- Encode those rules into a program

For example:

- “If the object has pointed ears and whiskers → classify as cat”
- “If the object has long snout and floppy ears → classify as dog”

At first glance, this might sound reasonable. But very quickly, we run into a serious problem.

Real-world images are not consistent. Lighting changes. Angles change. Backgrounds change. The same cat can look completely different depending on the situation. And suddenly, our neat collection of rules starts breaking down.

Even worse, rule-based systems have no real understanding of *patterns* or *relationships*. They only follow instructions we explicitly give them. So when a new image appears—something slightly different from what we expected—the system has no idea what to do. It cannot generalize. It cannot adapt.

1.2. The Shift: Learning from Data

This limitation pushed researchers toward a new idea: *Instead of writing rules manually, what if we let the machine learn the rules from data?*

So instead of saying: “*This is what a cat looks like*” → We say: “*Here are thousands of labeled examples of cats and dogs—figure it out yourself.*”

This is where traditional Machine Learning starts to appear. Models begin to learn patterns and relationships from data rather than relying on fixed human-defined logic. But even this approach has a limitation when we move into images.

The First Attempt: Traditional Neural Networks

Naturally, people tried to apply standard Neural Networks to images. At first, this seems promising:

- Images can be converted into numbers (pixels)
- Neural networks accept numerical input. So we simply feed the image into the network

But there is a hidden problem. A single image is not just “data”—it is **massive structured data**. For example, a small **256×256** RGB image already contains: $256 \times 256 \times 3 = 196,608$ input values (*Width = 256; Height = 256; Channels = 3*)

If we feed this directly into a fully connected neural network: Every pixel connects to every neuron. The number of parameters explodes. Training becomes extremely slow and inefficient

Even more importantly, a normal neural network does not understand spatial structure, treats every pixel as independent, and ignores the fact that nearby pixels are related.

But in images, structure is everything. Edges form shapes. Shapes form objects. Objects form meaning. A standard neural network simply cannot capture this hierarchy efficiently.

Why We Need Something Better: Convolutional Neural Networks

This is where Convolutional Neural Networks (CNNs) come in. Instead of treating an image as a flat vector, CNNs preserve spatial structure, focus on local patterns (like edges and textures), reuse filters across the entire image, reduce the number of parameters dramatically.

In other words, CNNs introduce a smarter way of looking at images—not as isolated pixels, but as structured visual patterns. They do not just “see” the image. They learn how to build understanding step by step:

- Edges → simple patterns
- Shapes → combinations of edges
- Objects → combinations of shapes

So at this point, we arrive at a clear progression:

- Rule-based systems → too rigid
- Traditional ML → better, but limited for raw images
- Fully connected neural networks → powerful, but inefficient for vision
- Convolutional Neural Networks → the right tool for the job

And now, we are ready to explore how CNNs actually work.

2. The Basics of Convolutional Neural Networks (CNNs)

From this point onward, we will use the abbreviation **CNN** (Convolutional Neural Network). Some books may refer to them as **ConvNets**, but they mean exactly the same thing.

In this chapter, our primary focus will be on **image classification problems**, particularly those involving relatively small datasets.

To demonstrate the concepts behind CNNs, we will use a familiar dataset: **MNIST**. By now, we have used MNIST multiple times throughout this course. However, this time we are not using it merely to train a model. Instead, we will use it to understand the mechanisms that make CNNs work.

Recall that each MNIST image is a grayscale image with dimensions: Height = 28 pixels; Width = 28 pixels; Channels = 1. Therefore, each input image has the shape: (28, 28, 1)

In general, CNNs expect image inputs in the form:

(height, width, channels)

For example:

- Grayscale image: (28, 28, 1);
- RGB image: (224, 224, 3)

A typical CNN architecture might look like this:

Conv2D → MaxPooling2D → Conv2D → MaxPooling2D → Conv2D → Flatten → Dense

At first glance, this may look complicated. But if we break CNNs apart, they are actually composed of two major components: The **Convolutional** part and the **Neural Network** part

2.1. The Convolutional Part: Extracting Features

The convolutional component is responsible for extracting useful features from an image. Instead of immediately asking: *"Is this image a cat or a dog?"*. The CNN first asks: *"What patterns can I find inside this image?"*

To accomplish this, CNNs use two major types of layers: Convolutional Layers and Pooling Layers. Inside these layers is a very important concept known as a **kernel** (sometimes called a filter).

A kernel is simply a small matrix that slides across the image and performs a mathematical operation known as **convolution**. Common kernel sizes include: 2×2 ; 3×3 ; 5×5 , although 3×3 kernels are by far the most common in modern CNNs.

2.2. Understanding the Convolution Operation

Rather than jumping directly into large images, let us begin with a very small example. Suppose our input image is represented by the matrix:

$$\begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix}$$

and suppose we use the following 2×2 kernel:

$$\begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix}$$

The convolution operation begins by placing the kernel on top of the first 2×2 region of the image:

$$\begin{bmatrix} 0 & 1 \\ 3 & 4 \end{bmatrix}$$

Now we multiply corresponding elements:

$$\begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix} \times \begin{bmatrix} 0 & 1 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 0 \times 0 & 1 \times 1 \\ 3 \times 2 & 4 \times 3 \end{bmatrix}$$

which gives:

$$\begin{bmatrix} 0 & 1 \\ 6 & 12 \end{bmatrix}$$

Finally, we sum all values: $0 + 1 + 6 + 12 = 19$. The result is a single scalar value:

19

This value becomes the first element of the output feature map. Next, the kernel slides one step to the right:

$$\begin{bmatrix} 1 & 2 \\ 4 & 5 \end{bmatrix}$$

Repeating the same process:

$$(1 \times 0) + (2 \times 1) + (4 \times 2) + (5 \times 3) = 0 + 2 + 8 + 15 = 25$$

This becomes the second output value. The kernel continues sliding across the image until every valid position has been processed.

The final output feature map becomes:

$$\begin{bmatrix} 19 & 25 \\ 37 & 43 \end{bmatrix}$$

At this point, you might be wondering: ***“Why perform all these calculations?”***. The answer is surprisingly important. Different kernels detect different visual patterns. Some kernels become good at detecting: Vertical edges; Horizontal edges; Corners; Textures; Shapes

As training progresses, the CNN automatically learns which kernels are most useful for the task. This means we no longer manually tell the computer what a cat’s ear looks like or what a dog’s nose looks like. Instead, the network learns those visual features directly from the training data.

2.3. The Neural Network Part

After several convolutional and pooling layers have extracted meaningful features, the CNN eventually reaches the traditional neural network component.

The extracted feature maps are flattened into a vector and passed into one or more **Dense layers**. These Dense layers behave exactly like the neural networks we studied earlier.

Their job is no longer to extract features. Instead, their job is to answer the final question:

"Given all the features discovered by the convolutional layers, what is the most likely class of this image?"

For MNIST, the answer would be one of the digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

So in short: **Convolutional layers** learn visual features. **Pooling layers** compress and preserve important information. **Dense layers** perform the final classification.

Together, these components form the foundation of a Convolutional Neural Network. In the next section, we will examine one of the most important operations in CNNs: **Pooling**.

3. Pooling Layers in Convolutional Neural Networks (CNNs)

Now we know that convolutional layers extract features from an image using kernels. However, after several convolution operations, the resulting feature maps can still be quite large. This is where **Pooling Layers** become useful.

Pooling is another matrix-based operation that works with small regions of the feature map. Instead of learning new features, pooling summarizes information by reducing the spatial size of the feature map.

This provides several benefits: Reduces computational cost; Reduces memory usage; Helps prevent overfitting; Preserves important information while discarding unnecessary details

The two most common pooling methods are Max Pooling and Average Pooling

3.1. Max Pooling

Max Pooling selects the **largest value** from each filtering region. Its purpose is to keep the strongest and most important features while reducing the size of the feature map.

Key characteristics of Max Pooling:

- Selects the largest value from each filter region
- Preserves important features such as edges and textures
- Reduces the spatial dimensions of the feature map
- Widely used in practice due to its strong performance

Let us consider the following feature map:

$$\begin{bmatrix} 2 & 2 & 7 & 3 \\ 9 & 4 & 6 & 1 \\ 8 & 5 & 2 & 4 \\ 3 & 1 & 2 & 6 \end{bmatrix}$$

Suppose we use a 2×2 pooling window.

The matrix is divided into four sub-matrices:

$$\begin{bmatrix} 2 & 2 \\ 9 & 4 \end{bmatrix} \quad \begin{bmatrix} 7 & 3 \\ 6 & 1 \end{bmatrix} \quad \begin{bmatrix} 8 & 5 \\ 3 & 1 \end{bmatrix} \quad \begin{bmatrix} 2 & 4 \\ 2 & 6 \end{bmatrix}$$

For each region, we select the maximum value:

$$\max \begin{bmatrix} 2 & 2 \\ 9 & 4 \end{bmatrix} = 9; \quad \max \begin{bmatrix} 7 & 3 \\ 6 & 1 \end{bmatrix} = 7; \quad \max \begin{bmatrix} 8 & 5 \\ 3 & 1 \end{bmatrix} = 8; \quad \max \begin{bmatrix} 2 & 4 \\ 2 & 6 \end{bmatrix} = 6$$

The resulting pooled feature map becomes:

$$\begin{bmatrix} 9 & 7 \\ 8 & 6 \end{bmatrix}$$

Notice that the original matrix had a size of 4×4 , while the pooled matrix now has a size of 2×2 . We reduced the amount of data by 75% while still preserving the strongest activations.

3.2. Average Pooling

Instead of selecting the largest value, Average Pooling computes the **average value** of each filtering region. This method focuses on the overall information contained within a region rather than only the strongest activation.

Key characteristics of Average Pooling:

- Computes the average value from each filter region
- Represents overall features instead of only dominant features
- Reduces the spatial dimensions of the feature map
- Produces smoother feature maps compared to Max Pooling

Using the same input matrix:

$$\begin{bmatrix} 2 & 2 & 7 & 3 \\ 9 & 4 & 6 & 1 \\ 8 & 5 & 2 & 4 \\ 3 & 1 & 2 & 6 \end{bmatrix}$$

Again, we divide it into four 2×2 regions.

For the first region:

$$\frac{2 + 2 + 9 + 4}{4} = \frac{17}{4} = 4.25$$

For the second region:

$$\frac{7 + 3 + 6 + 1}{4} = \frac{17}{4} = 4.25$$

For the third region:

$$\frac{8 + 5 + 3 + 1}{4} = \frac{17}{4} = 4.25$$

For the fourth region:

$$\frac{2 + 4 + 2 + 6}{4} = \frac{14}{4} = 3.5$$

The resulting feature map becomes:

$$\begin{bmatrix} 4.25 & 4.25 \\ 4.25 & 3.5 \end{bmatrix}$$

Max Pooling vs Average Pooling

Both methods reduce the size of feature maps, but they emphasize different information.

Max Pooling	Average Pooling
Keeps the strongest activation	Keeps the average activation
Highlights prominent features	Highlights overall patterns
Preserves edges and textures well	Produces smoother feature maps
Most commonly used in modern CNNs	Less common but still useful in some architectures

In practice, **Max Pooling is generally preferred** because it tends to preserve the most important visual features while aggressively reducing the size of the feature map. However, Average Pooling remains useful when we want to retain more global information about the image.

3. The Workflow of a Convolutional Neural Network

Now that we understand the individual components of a CNN, it is time to see how everything works together.

When people first hear the term *Convolutional Neural Network*, they often imagine a mysterious black box that somehow recognizes images. But underneath all the mathematics, a CNN follows a surprisingly logical workflow:

1. Extract small regions from the image.
2. Apply filters (kernels) to detect patterns.
3. Build feature maps that highlight useful information.
4. Learn which features matter most for classification.

In this section, we will walk through each of these steps manually before allowing TensorFlow to automate the process for us.

3.1. The Convolution Operation by Hand

Before we work with MNIST, let us understand what a convolution layer actually does. A convolution layer slides a small window, usually a 3×3 or 5×5 kernel, across an input image.

At every position:

- A small patch of the image is extracted.
- The patch is multiplied element-by-element with the kernel.
- The products are summed.
- The result becomes a value in the output feature map.

This process is repeated over the entire image.

An important idea here is **weight sharing**. Instead of learning a different set of weights for every location, the same kernel is reused everywhere in the image. This dramatically reduces the number of trainable parameters compared to a fully connected neural network.

The final result is an output tensor often called a **feature map**.

Suppose we have the following 5×5 image:

$$\begin{bmatrix} 1 & 2 & 3 & 0 & 1 \\ 4 & 5 & 6 & 1 & 2 \\ 7 & 8 & 9 & 2 & 3 \\ 0 & 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 & 5 \end{bmatrix}$$

and the kernel:

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

The kernel starts at the top-left corner and performs element-wise multiplication with the corresponding 3×3 patch.

The resulting value is placed into the output feature map. Then the kernel slides one step to the right and repeats the process. Eventually, the entire image is scanned.

Fortunately, TensorFlow can perform this operation for us.

```

import tensorflow as tf
import numpy as np

image = np.array([[1, 2, 3, 0, 1],
                  [4, 5, 6, 1, 2],
                  [7, 8, 9, 2, 3],
                  [0, 1, 2, 3, 4],
                  [1, 2, 3, 4, 5]], dtype="float32")

kernel = np.array([[1, 0, -1],
                  [1, 0, -1],
                  [1, 0, -1]], dtype="float32")

image = image.reshape((1, 5, 5, 1))
kernel = kernel.reshape((3, 3, 1, 1))

output = tf.nn.conv2d(image, kernel,
                      strides=1, padding="VALID")
print(output.numpy()[0, :, :, 0])

```

```

[[-6. 12. 12.]
 [-6.  8.  8.]
 [-6.  2.  2.]]

```

Notice that TensorFlow expects images in the format:

```
(batch_size, height, width, channels)
```

Even a single grayscale image must therefore be reshaped into a 4-dimensional tensor.

3.2. Feature Extraction on MNIST

Now let us move from toy examples to a real dataset. Throughout this course, MNIST has been our favorite playground. This time, however, we are not interested in the classification accuracy yet. Instead, we want to understand how CNNs extract information from images.

Some helper functions:

```

def plot_history(history, title="Training history"):
    metrics = history.history
    plt.figure(figsize=(11, 4))

    plt.subplot(1, 2, 1)
    plt.plot(metrics["accuracy"], "o-", label="Train")
    plt.plot(metrics["val_accuracy"], "o-", label="Validation")
    plt.title(title + " - Accuracy")
    plt.xlabel("Epoch")

```

```

plt.ylabel("Accuracy")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(metrics["loss"], "o-", label="Train")
plt.plot(metrics["val_loss"], "o-", label="Validation")
plt.title(title + " - Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.tight_layout()
plt.show()

def show_images(images, labels=None, class_names=None, n=10, cmap=None):
    plt.figure(figsize=(10, 3))
    for i in range(n):
        plt.subplot(2, 5, i + 1)
        if images.shape[-1] == 1:
            plt.imshow(images[i, :, :, 0], cmap=cmap or "gray")
        else:
            plt.imshow(np.clip(images[i], 0, 1), cmap=cmap)
        if labels is not None:
            label = int(labels[i])
            title = class_names[label] if class_names else str(label)
            plt.title(title)
        plt.axis("off")
    plt.tight_layout()
    plt.show()

```

We begin by loading the dataset.

```

from tensorflow import keras
from keras import layers

(train_images, train_labels), (test_images, test_labels) = (keras.datasets.
mnist.load_data())

```

Since neural networks work best with normalized values, we convert the pixel values from: 0 → 255 into: 0.0 → 1.0

```

# Normalized
train_images = train_images.astype("float32") / 255.0
test_images = test_images.astype("float32") / 255.0

```

Next, we add a channel dimension:

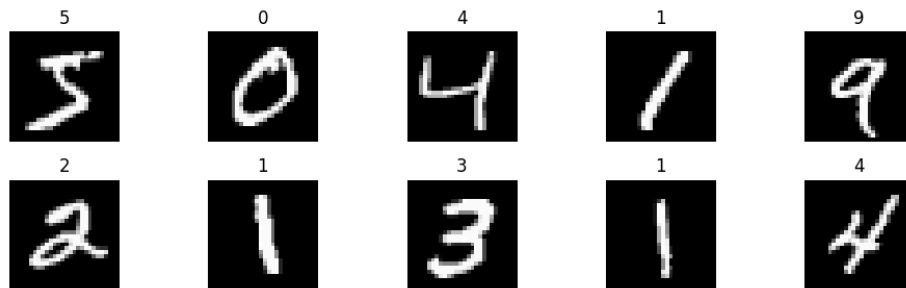
```
# Add a channel dimension
train_images = np.expand_dims(train_images, axis=-1)
test_images = np.expand_dims(test_images, axis=-1)

# Return shape
print(train_images.shape)
print(test_images.shape)
```

```
(60000, 28, 28, 1)
(10000, 28, 28, 1)
```

The resulting shape becomes: $(samples, 28, 28, 1)$, which is exactly what a CNN expects. After creating training and validation sets, we can visualize a few examples:

```
show_images(train_images, train_labels, n=10, cmap="gray")
```



At this point, each image is simply a 28×28 matrix of pixel values. But here comes the important question: **What exactly does a convolution layer see?**

The answer is surprisingly simple. It does not see the entire image at once. Instead, it repeatedly extracts small local regions from the image. In this example, we use a 5×5 window, which corresponds to the size of the convolution kernel. By sliding this window across the image one pixel at a time, we can observe exactly what the convolution layer processes at each location.

```
image = train_images[0, :, :, 0]

patches = []

for row in range(24):
    for col in range(24):
        patch = image[row:row+5, col:col+5]
        patches.append(patch)

print("Number of 5x5 patches:", len(patches))
```

```
Number of 5x5 patches: 576
```

Why 576? Because: $(28 - 5 + 1) \times (28 - 5 + 1) = 24 \times 24 = 576$

This means a single 28×28 MNIST image contains 576 different 5×5 regions that a convolution layer can inspect. At every location, the same kernel weights are applied to the current 5×5 patch, producing a single value in the output feature map.

Reusing the same kernel across all locations is known as **weight sharing**, one of the key ideas that makes Convolutional Neural Networks efficient and effective.

Optionally, to visualize what the kernel sees, we can display the first 16 extracted patches as enlarged images by plotting **where the kernel is located on the original image**:

```
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle

fig, axes = plt.subplots(4, 4, figsize=(10, 10))

patch_num = 1

for row in range(4):
    for col in range(4):
        ax = axes[row, col]

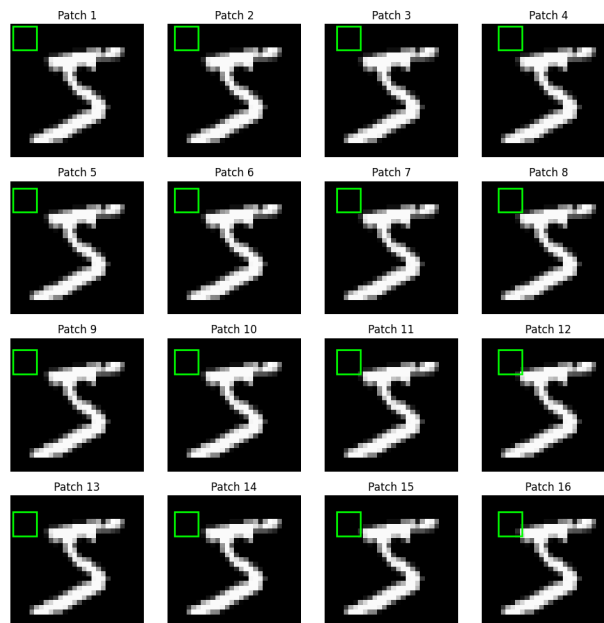
        ax.imshow(image, cmap='gray')

        rect = Rectangle((col, row), # x, y
                        5,           # width
                        5,           # height
                        linewidth=2,
                        edgecolor='lime',
                        facecolor='none')

        ax.add_patch(rect)
        ax.set_title(f"Patch {patch_num}")
        ax.axis('off')

        patch_num += 1

plt.tight_layout()
plt.show()
```



Now you can literally see the **5×5 convolution window sliding across the image**.



It is important to note that these highlighted regions are **not yet the feature map**. They are simply the local 5×5 patches (also called receptive fields) that the convolution layer examines.

- For each patch, the kernel performs an element-wise multiplication, sums the results, and produces a single output value.
- As the kernel slides across all possible positions in the image, these output values are arranged into a new matrix called the **feature map** (or **response map**).
- For a 28×28 image and a 5×5 kernel with stride 1 and no padding, there are 576 valid kernel positions.
- Consequently, the convolution operation produces a 24×24 feature map, where each value summarizes the information extracted from one 5×5 region of the original image.
- This feature map is then passed to subsequent layers for further processing.

```

kernel = np.array([
    [1, 0, -1, 0, 1],
    [1, 0, -1, 0, 1],
    [1, 0, -1, 0, 1],
    [1, 0, -1, 0, 1],
    [1, 0, -1, 0, 1]
])

feature_map = np.zeros((24, 24))

for row in range(24):
    for col in range(24):
        patch = image[row:row+5, col:col+5]
        feature_map[row, col] = np.sum(patch * kernel)

```

Now `feature_map` is the actual **response map** / **feature map** produced by the convolution.

- The 576 extracted 5×5 regions → **patches, local regions, or receptive fields**.
- The 24×24 output after applying the kernel → **feature map** (or **response map**).

3.3. Filtering, Response Maps, and Feature Maps

Now we arrive at the heart of CNNs. A kernel acts like a detector. Different kernels respond strongly to different visual patterns. For example: Vertical edges; Horizontal edges; Corners; Curves; Textures.

Let us create two handcrafted kernels.

```

vertical_edge_kernel = np.array([[-1, 0, 1],
                                  [-1, 0, 1],
                                  [-1, 0, 1]],
                                dtype="float32")

horizontal_edge_kernel = np.array([[-1, -1, -1],
                                    [0, 0, 0],
                                    [1, 1, 1]],
                                   dtype="float32")

```

The first kernel detects vertical transitions. The second detects horizontal transitions.

Applying them to an image:

```

# Use [0:1] to preserve the batch size dimension
# (i.e., (1, 28, 28, 1) instead of (28, 28, 1))
sample = test_images[0:1]

```

```

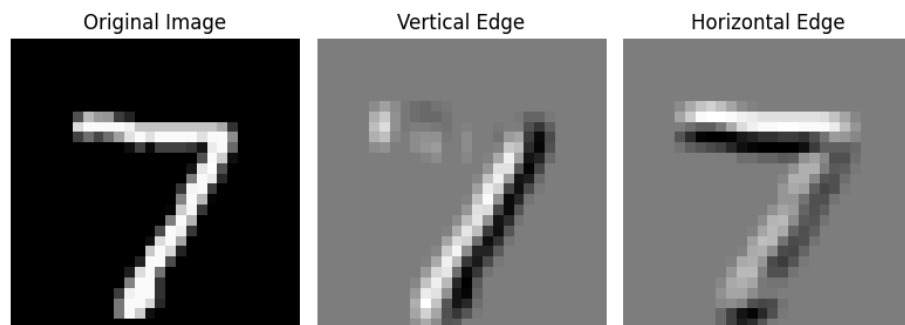
kernels = {"Vertical Edge": vertical_edge_kernel,
           "Horizontal Edge": horizontal_edge_kernel,}

plt.figure(figsize=(8, 3))
plt.subplot(1, 3, 1)
plt.imshow(sample[0, :, :, 0], cmap="gray")
plt.title("Original Image")
plt.axis("off")

for i, (name, kernel) in enumerate(kernels.items(), start=2):
    kernel_4d = kernel.reshape((3, 3, 1, 1))
    response = tf.nn.conv2d(sample, kernel_4d, strides=1, padding="SAME").
    numpy()
    plt.subplot(1, 3, i)
    plt.imshow(response[0, :, :, 0], cmap="gray")
    plt.title(name)
    plt.axis("off")

plt.tight_layout()
plt.show()

```



This produces a new image. Such new image is called a **response map because it shows how strongly the kernel responds at every spatial location in the image.**

The brighter a region appears in the response map, the stronger that kernel detected its target pattern. For example, vertical-edge kernels light up vertical strokes, and horizontal-edge kernels light up horizontal strokes. This is why a handwritten digit such as "1" produces a strong response for vertical-edge detectors.

From Response Maps to Feature Maps

The terms **response map** and **feature map** are often used interchangeably in CNN literature. In this section, we will use the following distinction:

- A **response map** refers to the output generated by a single kernel.
- A **feature map tensor** refers to the collection of response maps produced by all kernels in a convolution layer.

For example, if a convolution layer contains 16 kernels, then:

- Kernel 1 produces Response Map 1
- Kernel 2 produces Response Map 2
- ...
- Kernel 16 produces Response Map 16

Stacking these 16 response maps along the channel dimension produces a feature map tensor with shape $(1, 28, 28, 16)$, where:

- 1 = batch size
- 28 = height
- 28 = width
- 16 = channels (one channel per learned kernel)

Visually:

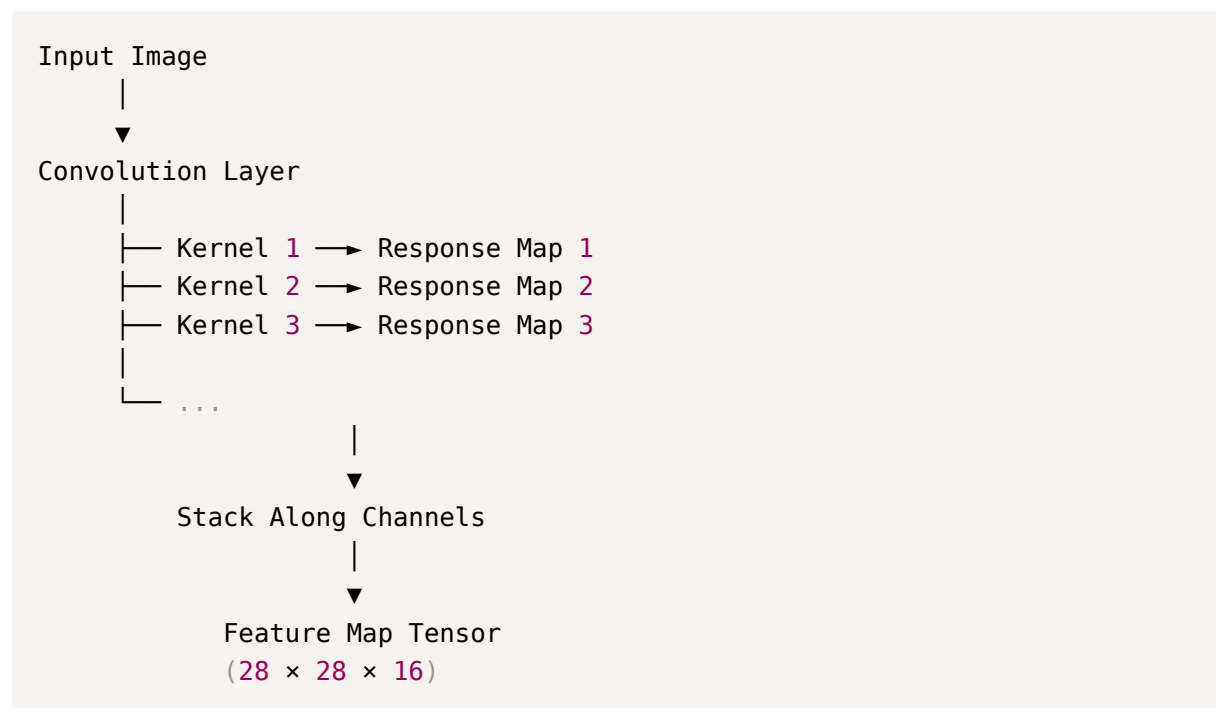


Figure 8.1 Images can be broken into local patterns such as edges, textures, and so on.

Feature Map

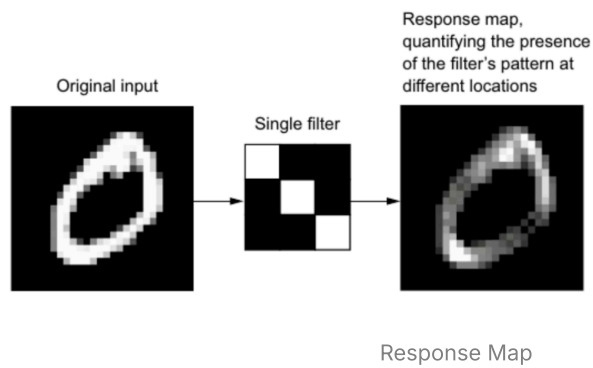


Figure 8.3 The concept of a response map: a 2D map of the presence of a pattern at different locations in an input

Each channel corresponds to a different learned detector. Some channels may respond strongly to vertical lines, others to horizontal edges, curves, loops, corners, or more complex visual structures. The remarkable part is that we never explicitly teach the CNN what these patterns are. The network discovers them automatically during training.

This is the true power of deep learning. Instead of manually engineering features, we allow the model to learn which visual patterns are useful for solving the problem.

Now we can naturally continue with building the full CNN architecture (`Conv2D` → `MaxPooling2D` → `Conv2D` → `Flatten` → `Dense`), compiling it, training it, and analyzing the learning curves.

3.4. Training the Convolutional Neural Network

At this point, we are no longer just observing how convolution works at a local level. We now move to something more complete: a full model that can actually learn from these patterns.

If convolution layers are about *seeing local structure*, then training is about teaching the network to *understand what those structures mean*.

Let's build the CNN.

```
def build_mnist_convnet():
    inputs = keras.Input(shape=(28, 28, 1))

    # Convolutional feature extraction
    x = layers.Conv2D(filters=32, kernel_size=3, activation="relu", name="conv_1")(inputs)
    x = layers.MaxPooling2D(pool_size=2, name="pool_1")(x)

    x = layers.Conv2D(filters=64, kernel_size=3, activation="relu", name="conv_2")(x)
    x = layers.MaxPooling2D(pool_size=2, name="pool_2")(x)

    x = layers.Conv2D(filters=128, kernel_size=3, activation="relu", name="conv_3")(x)

    # Classification head
    x = layers.Flatten()(x)
    outputs = layers.Dense(10, activation='softmax')(x)
```

```
model = keras.Model(inputs=inputs, outputs=outputs)
return model
```

Here, something important is happening:

- Early layers (`conv_1`) learn simple patterns like edges
- Middle layers (`conv_2`) combine those patterns into shapes
- Deeper layers (`conv_3`) start forming higher-level abstractions

Each layer is not just transforming data—it is progressively refining *what the image means*.

Now we define how the model learns.

```
mnist_convnet = build_mnist_convnet()

mnist_convnet.compile(optimizer="rmsprop",
                      loss="sparse_categorical_crossentropy",
                      metrics=["accuracy"],)
mnist_convnet.summary()
```

At this stage, the model is still untrained. It is only a structure—an architecture waiting to be shaped by data.

Now we allow the model to learn from experience.

```
FAST_RUN = True

epochs = 2 if FAST_RUN else 5

history_mnist = mnist_convnet.fit(train_images, train_labels,
                                  epochs=epochs,
                                  validation_data=(test_images, test_labels))
```

During training, something subtle but powerful happens:

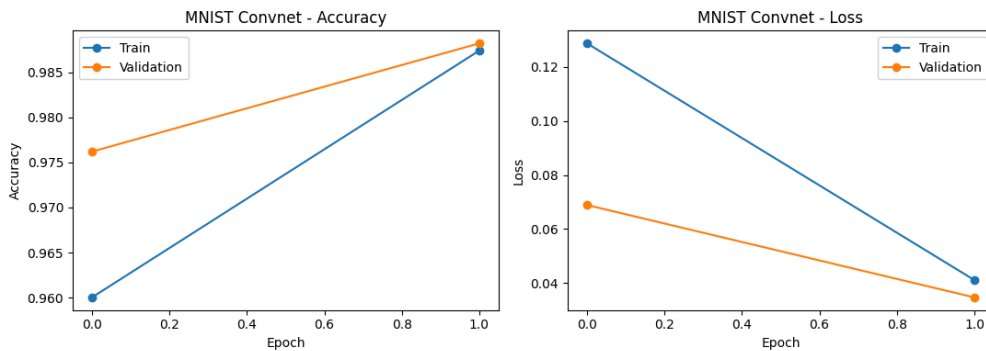
- Filters inside convolution layers adjust themselves
- Patterns that help classification become stronger
- Irrelevant signals gradually fade

What begins as random weights slowly becomes a structured feature detector.

Evaluation: How Well Did It Learn?

```
test_loss, test_acc = mnist_convnet.evaluate(test_images, test_labels, verbose=0)
print(f"MNIST test accuracy: {test_acc:.4f}")
```

```
plot_history(history_mnist, "MNIST Convnet")
```



At this point, we are no longer asking “*can it learn?*”. We are asking “*can it generalize?*”. Because learning alone is not enough—the model must perform on data it has never seen before.

Comparing with a Dense Model

To understand the value of convolution, we compare it with a simpler approach. A dense model does not see spatial structure. It flattens everything into a single vector.

```
def build_dense_mnist_model():
    inputs = keras.Input(shape=(28, 28, 1))

    x = layers.Flatten()(inputs)
    x = layers.Dense(256, activation="relu")(x)
    outputs = layers.Dense(10, activation="softmax")(x)

    model = keras.Model(inputs, outputs)
    return model
```

Here, something important is missing:

- No local perception
- No spatial awareness
- No concept of “edges” or “regions”

Everything is treated as one long list of numbers.

Training the Dense Model

```
dense_model = build_dense_mnist_model()

dense_model.compile(optimizer="rmsprop",
                    loss="sparse_categorical_crossentropy",
```

```

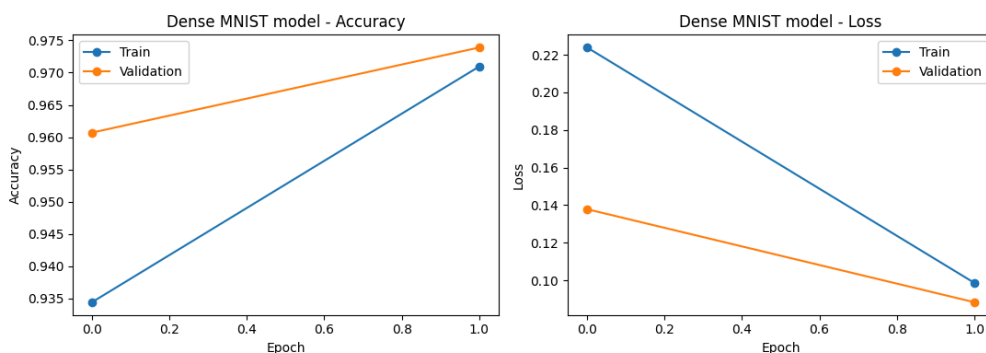
metrics=["accuracy"],)

history_dense = dense_model.fit(train_images, train_labels,
                                epochs=epochs,
                                validation_data=(test_images, test_labels),)

dense_test_loss, dense_test_acc = dense_model.evaluate(test_images, test_labels, verbose=0)
print(f"Dense test accuracy: {dense_test_acc:.4f}")

plot_history(history_dense, "Dense MNIST model")

```



Now we can observe a key idea: CNNs learn structure. Dense networks learn correlations. And structure is usually more powerful when dealing with images.

Understanding What the CNN Sees

To make this more concrete, we inspect what the first convolution layer has learned.

```

activation_model = keras.Model(inputs=mnist_convnet.input,
                               outputs=mnist_convnet.get_layer("conv_1").output,)

activations = activation_model.predict(test_images[0:1], verbose=0)
print("Activation shape:", activations.shape)

```

This model does not predict digits. Instead, it reveals perception.

```

activations = activation_model.predict(test_images[0:1], verbose=0)
print("Activation shape:", activations.shape)

```

Each channel here is a **response map**—a different way the network “reacts” to the same image.

Visualizing Feature Maps

```
plt.figure(figsize=(10, 6))

for i in range(16):
    plt.subplot(4, 4, i + 1)
    plt.imshow(activations[0, :, :, i], cmap="viridis")
    plt.title(f"Map {i}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Activation shape: (1, 26, 26, 32)

Explanation:

- 1: Number of samples (here, 1 test image).
- 26, 26: Height and width of the feature map.
- 32: Number of feature maps (or the number of filters/channels) in the conv_1 layer."

What you are seeing is not the image itself anymore. It is the image as interpreted by learned filters. Each map highlights a different kind of structure. Some detect edges. Some respond to curves. Some activate on strokes. Some remain almost silent

Together, they form a distributed perception system.

At this point, the distinction becomes clear:

- The **input image** is raw information
- The **feature maps** are interpreted signals
- The **dense layer** makes the final decision

And between these steps lies the essence of CNNs: Not seeing pixels as pixels, but as patterns arranged in space. This is where the model stops being a formula and starts behaving like a perception system.

4. Image Classification with Convolutional Neural Networks

If MNIST was the "first conversation" with CNNs, then this section is where things become more realistic. Because real-world images are no longer clean, centered digits. They contain noise, variation, background, and ambiguity.

So instead of handwritten digits, we now move to something more natural: **Cats vs Dogs**.

4.1. Cat vs Dog Dataset (CIFAR-10 subset)

We start by loading CIFAR-10 and isolating only the two classes we care about.

```
(cifar_train, cifar_y_train), (cifar_test, cifar_y_test) = keras.datasets.
cifar10.load_data()

cifar_y_train = cifar_y_train.squeeze()
cifar_y_test = cifar_y_test.squeeze()

CAT_LABEL = 3
DOG_LABEL = 5
```

It may take a while for the system to fully download the dataset

Now we filter the dataset. Instead of using all 10 classes, we reduce the problem into a simpler binary task: Cat → 0; Dog → 1

```
def filter_cat_dog(images, labels):
    mask = np.isin(labels, [CAT_LABEL, DOG_LABEL])
    images = images[mask].astype("float32") / 255.0
    labels = labels[mask]
    labels = (labels == DOG_LABEL).astype("float32")
    return images, labels
```

We apply this transformation to both training and test sets:

```
cd_images, cd_labels = filter_cat_dog(cifar_train, cifar_y_train)
cd_test_images, cd_test_labels = filter_cat_dog(cifar_test, cifar_y_test)

print("Cat/Dog train pool:", cd_images.shape, cd_labels.shape)
print("Cat/Dog test:", cd_test_images.shape, cd_test_labels.shape)
```

Cat/Dog train pool: (10000, 32, 32, 3) (10000,)

Cat/Dog test: (2000, 32, 32, 3) (2000,)

And we take a quick look at the data:

```
show_images(cd_images, cd_labels, class_names={0: "cat", 1: "dog"}, n=10)
```

Building a Balanced Dataset

A model is only as good as the data distribution it sees. So we construct a balanced subset.

```
SEED = 42
np.random.seed(SEED)
tf.random.set_seed(SEED)

n_per_class_train = 700 if FAST_RUN else 2000
n_per_class_val = 200 if FAST_RUN else 500
```

```
cat_indices = np.where(cd_labels == 0)[0]
dog_indices = np.where(cd_labels == 1)[0]

rng = np.random.default_rng(SEED)
rng.shuffle(cat_indices)
rng.shuffle(dog_indices)
```

We then split into training and validation sets:

```
train_indices = np.r_[cat_indices[:n_per_class_train],
                     dog_indices[:n_per_class_train],]

val_indices = np.r_[cat_indices[n_per_class_train:n_per_class_train
                              + n_per_class_val],
                  dog_indices[n_per_class_train:n_per_class_train
                              + n_per_class_val],]

rng.shuffle(train_indices)
rng.shuffle(val_indices)
```

Finally, we extract the actual tensors:

```
x_cd_train = cd_images[train_indices]
y_cd_train = cd_labels[train_indices]

x_cd_val = cd_images[val_indices]
y_cd_val = cd_labels[val_indices]

print("x_cd_train:", x_cd_train.shape)
print("x_cd_val:", x_cd_val.shape)
print("cd_test_images:", cd_test_images.shape)
```

```
x_cd_train: (1400, 32, 32, 3)
x_cd_val: (400, 32, 32, 3)
cd_test_images: (2000, 32, 32, 3)
```

At this point, we are ready to train a model that sees something closer to the real world.

4.2. Baseline Convolutional Model

We begin with a simple CNN—our baseline. This model is not complex, but it already contains the essential idea: **Local feature extraction** → **Spatial compression** → **Classification**

```
def build_catdog_baseline():
    inputs = keras.Input(shape=(32, 32, 3))

    x = layers.Conv2D(32, 3, activation="relu")(inputs)
```



```

x = layers.MaxPooling2D(2)(x)

x = layers.Conv2D(64, 3, activation="relu")(x)
x = layers.MaxPooling2D(2)(x)

x = layers.Conv2D(128, 3, activation="relu")(x)

x = layers.Flatten()(x)
outputs = layers.Dense(1, activation="sigmoid")(x)

model = keras.Model(inputs, outputs)
model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"],)

return model

```

We initialize and inspect the model:

```

catdog_baseline = build_catdog_baseline()
catdog_baseline.summary()

```

Training the Baseline

```

epochs_cd = 4 if FAST_RUN else 20

history_cd = catdog_baseline.fit(x_cd_train, y_cd_train,
                                epochs=epochs_cd, batch_size=64,
                                validation_data=(x_cd_val, y_cd_val),)

```

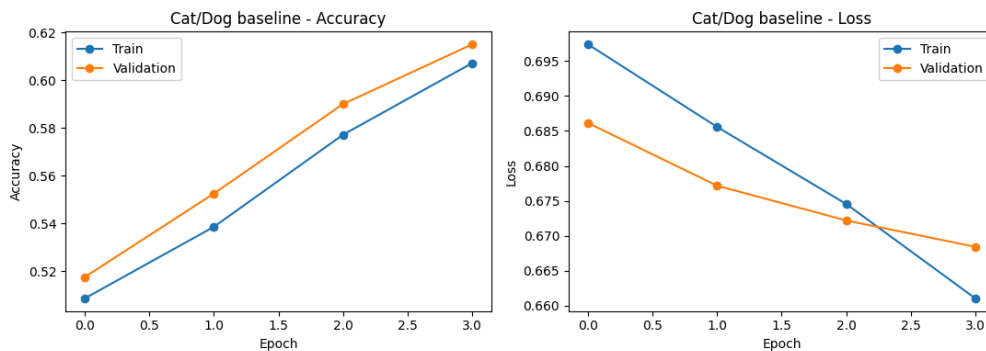
Now the model is no longer “just a structure”. It is something that has started to *adapt*. We evaluate it:

```

test_loss, test_acc = catdog_baseline.evaluate(cd_test_images, cd_test_labels, verbose=0)
print(f"Baseline Cat/Dog test accuracy: {test_acc:.4f}")

plot_history(history_cd, "Cat/Dog baseline")

```



4.3. Improving the Model: Augmentation + Dropout

At this stage, we confront a common reality: A model is often limited not by architecture, but by data. Two key tools help us here are **Augmentation** and **Dropout**

- **Data Augmentation:** Instead of collecting new data, we *transform existing data*.

```
data_augmentation = keras.Sequential([layers.RandomFlip("horizontal"),
                                     layers.RandomRotation(0.08),
                                     layers.RandomZoom(0.15)],
                                     name="data_augmentation",)
```

We can visualize what this means:

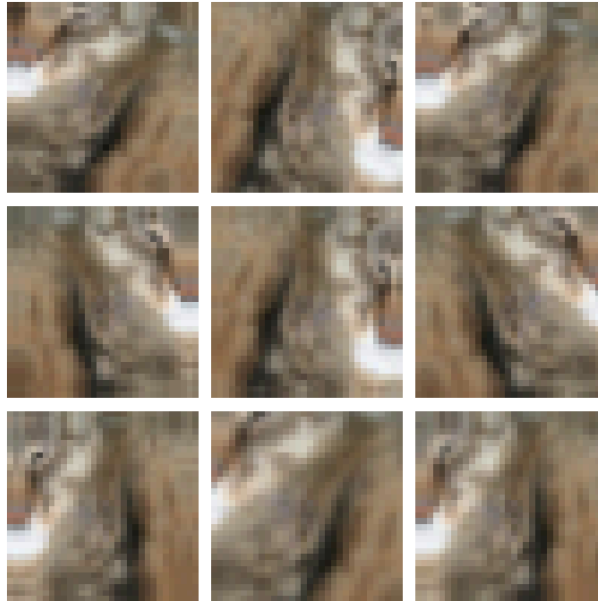
```
sample_img = x_cd_train[0:1]

plt.figure(figsize=(7, 7))

for i in range(9):
    augmented = data_augmentation(sample_img, training=True)

    plt.subplot(3, 3, i + 1)
    plt.imshow(np.clip(augmented[0].numpy(), 0, 1))
    plt.axis("off")

plt.tight_layout()
plt.show()
```



Each transformation creates a slightly different version of the same image. Not new information—but new perspectives.

- **Dropout:** If augmentation increases diversity in data, dropout increases uncertainty in the model. It prevents the network from relying too heavily on any single neuron.

```
def build_catdog_augmented():
    inputs = keras.Input(shape=(32, 32, 3))

    x = data_augmentation(inputs)

    x = layers.Conv2D(32, 3, activation="relu")(x)
    x = layers.MaxPooling2D(2)(x)

    x = layers.Conv2D(64, 3, activation="relu")(x)
    x = layers.MaxPooling2D(2)(x)

    x = layers.Conv2D(128, 3, activation="relu")(x)

    x = layers.Flatten()(x)

    x = layers.Dropout(0.5)(x)

    outputs = layers.Dense(1, activation="sigmoid")(x)

    model = keras.Model(inputs, outputs)

    model.compile(optimizer="rmsprop", loss="binary_crossentropy",
                  metrics=["accuracy"],)
```

```
return model
```

We train again:

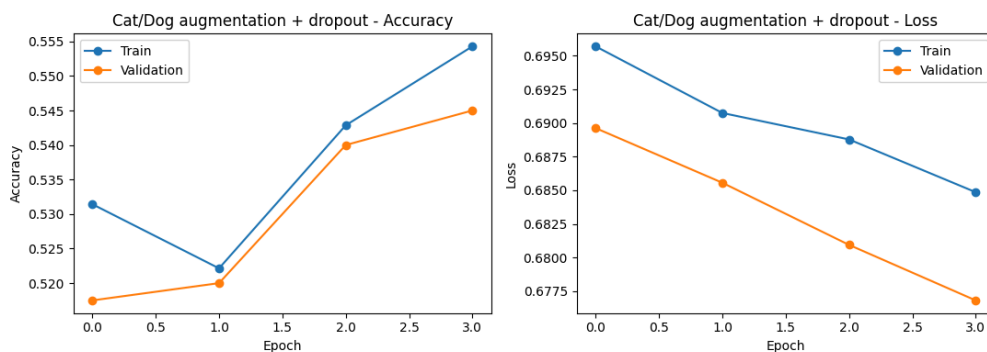
```
catdog_augmented = build_catdog_augmented()
catdog_augmented.summary()

history_aug = catdog_augmented.fit(x_cd_train,
                                    y_cd_train,
                                    epochs=4 if FAST_RUN else 30,
                                    batch_size=64,
                                    validation_data=(x_cd_val, y_cd_val),)
```

And evaluate:

```
test_loss, test_acc = catdog_augmented.evaluate(cd_test_images, cd_test_labels, verbose=0)
print(f"Augmented Cat/Dog test accuracy: {test_acc:.4f}")

plot_history(history_aug, "Cat/Dog augmentation + dropout")
```



Summary

At this point, the progression becomes clear: We started with a simple CNN that learns local patterns. We moved to real-world images with more complexity. Then we improved generalization using augmentation and dropout.

And what we see is a consistent pattern: **CNNs are not just about deeper layers.** They are about learning representations that survive variation.

Or more simply: A good model is not the one that memorizes cats and dogs. It is the one that still recognizes them when everything looks slightly different.

References

<https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-convolutional-neural-networks/>

<https://www.geeksforgeeks.org/machine-learning/introduction-convolution-neural-network/>

<https://www.datacamp.com/tutorial/introduction-to-convolutional-neural-networks-cnns>

<https://www.ibm.com/think/topics/convolutional-neural-networks>

Next Section:

 [14. Pre-Trained Neural Network Models](#)